下载APP

加餐｜愚昧之巅：你的Rust学习常见问题汇总

2021-09-17 陈天

《陈天 · Rust 编程第一课》 课程介绍 >



讲述：陈天

时长 12:45 大小 11.69M

你好，我是陈天。

到目前为止，我们已经学了很多 Rust 的知识，比如基本语法、内存管理、所有权、生命周期等，也展示了三个非常有代表性的示例项目，让你了解接近真实应用环境的 Rust 代码是什么样的。

虽然学了这么多东西，你是不是还是有种“一学就会，一写就废”的感觉？别着急，饭要一口一口吃，任何新知识的学习都不是一蹴而就的，我们让子弹先飞一会。你也可以鼓励一下自己，已经完成了这么多次打卡，继续坚持。

在今天这个加餐里我们就休个小假，调整一下学习节奏，来聊一聊 Rust 开发中的常见问题，希望可以解决你的一些困惑。

所有权问题

Q：如果我想创建双向链表，该怎么处理？

Rust 标准库有 LinkedList，它是一个双向链表的实现。但是当你需要使用链表的时候，可以先考虑一下，**同样的需求是否可以用列表 Vec<T>、循环缓冲区 VecDeque<T> 来实现**。因为，链表对缓存非常不友好，性能会差很多。

如果你只是好奇如何实现双向链表，那么可以用之前讲的 Rc / RefCell（第 9 讲）来实现。对于链表的 next 指针，你可以用 Rc；对于 prev 指针，可以用 Weak。

Weak 相当于一个弱化版本的 Rc，不参与到引用计数的计算中，而 Weak 可以 upgrade 到 Rc 来使用。如果你用过其它语言的引用计数数据结构，你应该对 Weak 不陌生，它可以帮我们打破循环引用。感兴趣的同学可以自己试着实现一下，然后对照这个参考实现。

你也许好奇为什么 Rust 标准库的 LinkedList 不用 Rc/Weak，那是因为标准库直接用 NonNull 指针和 unsafe。

Q：编译器总告诉我：“use of moved value” 错误，该怎么破？

这是我们初学 Rust 时经常会遇到的错误，这个错误是说**你在试图访问一个所有权已经移走的变量**。

对于这样的错误，首先你要判断，这个变量真的需要被移动到另一个作用域下么？如果不需要，可不可以使用借用？（第 8 讲）如果的确需要移动给另一个作用域的话：

1. 如果需要多个所有者共享同一份数据，可以使用 Rc / Arc，辅以 Cell / RefCell / Mutex / RwLock。（第 9 讲）
2. 如果不需要多个所有者共享，那可以考虑实现 Clone 甚至 Copy。（第 7 讲）

生命周期问题

Q：为什么我的函数返回一个引用的时候，编译器总是跟我过不去？

函数返回引用时，除非是静态引用，那么这个引用一定和带有引用的某个输入参数有关。输入参数可能是 &self、&mut self 或者 &T / &mut T。**我们要建立正确的输入和返回值之间的关系，这个关系和函数内部的实现无关，只和函数的签名有关。**

比如 HashMap 的 get() 方法：

复制代码

```
pub fn get<Q: ?Sized>(&self, k: &Q) -> Option<&V>
    where
        K: Borrow<Q>,
        Q: Hash + Eq
```

我们并不用实现它或者知道它如何实现，就可以确定返回值 Option<&V> 到底跟谁有关系。因为这里只有两个选择：&self 或者 k: &Q。显然是 &self，因为 HashMap 持有数据，而 k 只是用来在 HashMap 里查询的 key。

这里为什么不需要使用生命周期参数呢？因为我们之前讲的规则：**当 &self / &mut self 出现时，返回值的生命周期和它关联。**（第 10 讲）这是一个很棒的规则，因为大部分方法，如果返回引用，它基本上是引用 &self 里的某个数据。

如果你能搞明白这一层关系，那么就比较容易处理，函数返回引用时出现的生命周期错误。

当你要返回在函数执行过程中，创建的或者得到的数据，和参数无关，那么**无论它是一个有所有权的数据，还是一个引用，你只能返回带所有权的数据。**对于引用，这就意味着调用 clone() 或者 to_owned() 来，从引用中得到所有权。

数据结构问题

Q：为什么 Rust 字符串这么混乱，有 String、&String、&str 这么多不同的表述？

我不得不说，这是一个很有误导性的问题，因为这个问题有点胡乱总结的倾向，很容易把人带到沟里。

首先，任何数据结构 T，都可以有指向它的引用 &T，**所以 String 跟 &String 的区别，以及 String 跟 &str 的区别，压根是两个问题。**

更好的问题是：为什么有了 String，还要有 &str？或者，更通用的问题：为什么 String、Vec<T> 这样存放连续数据的容器，还要有切片的概念呢？

一旦问到点子上，答案不言自喻，因为切片是一个非常通用的数据结构。

用过 Python 的人都知道：

```
复制代码
1 s = "hello world"
2 let slice1 = s[:5] # 可以对字符串切片
3 let slice2 = slice1[1:3] # 可以对切片再切片
4 print(slice1, slice2) # 打印 hello, el
```

这和 Rust 的 String 切片何其相似：

```
复制代码
1 let s = "hello world".to_string();
2 let slice1 = &s[..5]; // 可以对字符串切片
3 let slice2 = &slice1[1..3]; // 可以对切片再切片
4 println!("{} {}", slice1, slice2); // 打印 hello el
```

所以 &str 是 String 的切片，也可以是 &str 的切片。它和 &[T] 一样，没有什么特别的，就是一个带着长度的胖指针，指向了一片连续的内存区域。

你可以这么理解：**切片之于 Vec<T> / String 等数据，就好比数据库里的视图（view）之于表（table）**。关于这个问题我们会在后面，讲 Rust 的数据结构时详细讲到。

Q：在课程的示例代码中，用了很多 unwrap()，这样可以么？

当我们需要从 Option或者 Result<T, E> 中获得数据时，可以使用 unwrap()，这是示例代码出现 unwrap() 的原因。

如果我们只是写一些学习性质的代码，那么 unwrap() 是可以接受的，但在生产环境中，除非你可以确保 unwrap() 不会引发 panic!()，否则应该使用模式匹配来处理数据，或者使用错误处理的 ? 操作符。我们后续会有专门一讲聊 Rust 的错误处理。

那什么情况下我们可以确定 unwrap() 不会 panic 呢？如果在做 unwrap() 之前，**Option<T> 或者 Result<T, E> 中已经有合适的值（Some(T) 或者 Ok(T)）**，你就可以做 unwrap()。比如这样的代码：

复制代码

```
// 假设 v 是一个 Vec<T>
if v.is_empty() {
    return None;
}

// 我们现在确定至少有一个数据，所以 unwrap 是安全的
let first = v.pop().unwrap();
```

Q：为什么标准库的数据结构比如 Rc / Vec 用那么多 unsafe，但别人总是告诉我，unsafe 不好？

好问题。C 语言的开发者也认为 asm 不好，但 C 的很多库里也大量使用 asm。

标准库的责任是，在保证安全的情况下，即使牺牲一定的可读性，也要用最高效的手段来实现要实现的功能；同时，为标准库的用户提供一个优雅、高级的抽象，让他们可以在绝大多数场合下写出漂亮的代码，无需和丑陋打交道。

Rust 中，unsafe 代码把程序的正确性和安全性交给了开发者来保证，而标准库的开发者花了大量的精力和测试来保证这种正确性和安全性。而我们自己撰写 unsafe 代码时，除非有经验丰富的开发者 review 代码，否则，有可能疏于对并发情况的考虑，写出了有问题的代码。

所以只要不是必须，建议不要写 unsafe 代码。**毕竟大部分我们要处理的问题，都可以通过良好的设计、合适的数据结构和算法来实现。**

Q：在 Rust 里，我如何声明全局变量呢？

在第 3 讲里，我们讲过 const 和 static，它们都可以用于声明全局变量。但注意，除非使用 unsafe，static 无法作为 mut 使用，因为这意味着它可能在多个线程下被修改，所以不安全：

复制代码

```
static mut COUNTER: u64 = 0;

fn main() {
    COUNTER += 1; // 编译不过，编译器告诉你需要使用 unsafe
}
```

如果你的确想用可写的全局变量，可以用 Mutex<T>，然而，初始化它很麻烦，这时，你可以用一个库 lazy_static。比如（代码）：

复制代码

```
use lazy_static::lazy_static;
use std::collections::HashMap;
use std::sync::{Arc, Mutex};

lazy_static! {
    static ref HASHMAP: Arc<Mutex<HashMap<u32, &'static str>>> = {
        let mut m = HashMap::new();
        m.insert(0, "foo");
        m.insert(1, "bar");
        m.insert(2, "baz");
        Arc::new(Mutex::new(m))
    };
}

fn main() {
    let mut map = HASHMAP.lock().unwrap();
    map.insert(3, "waz");

    println!("map: {:?}", map);
}
```

调试工具

Q：Rust 下，一般如何调试应用程序？

我自己一般会用 tracing 来打日志，一些简单的示例代码会使用 println! / dbg! ，来查看数据结构在某个时刻的状态。而在平时的开发中，我几乎不会用调试器设置断点单步跟踪。

因为与其浪费时间在调试上，不如多花时间做设计。**在实现的时候，添加足够清晰的日志，以及撰写合适的单元测试，来确保代码逻辑上的正确性**。如果你发现自己总需要使用调试工具单步跟踪才能搞清楚程序的状态，说明代码没有设计好，过于复杂。

当我学习 Rust 时，会常用调试工具来查看内存信息，后续的课程中我们会看到，在分析有些数据结构时使用了这些工具。

Rust 下，我们可以用 rust-gdb 或 rust-lldb，它们提供了一些对 Rust 更友好的 pretty-print 功能，在安装 Rust 时，它们也会被安装。我个人习惯使用 gdb，但 rust-gdb 适合在 linux 下，在 OS X 下有些问题，所以我一般会切到 Ubuntu 虚拟机中使用 rust-gdb。

其它问题

Q：为什么 Rust 编译出来的二进制那么大？为什么 Rust 代码运行起来那么慢？

如果你是用 cargo build 编译出来的，那很正常，因为这是个 debug build，里面有大量的调试信息。你可以用 cargo build --release 来编译出优化过的版本，它会小很多。另外，还可以通过很多方法进一步优化二进制的大小，如果你对此感兴趣，可以参考这个文档。

Rust 的很多库如果你不用 --release 来编译，它不会做任何优化，有时候甚至感觉比你的 Node.js 代码还慢。所以当你要把代码应用在生产环境，一定要使用 release build。

Q：这门课使用什么样的 Rust 版本？会随着 2021 edition 更新么？

会的。Rust 是一门不断在发展的语言，每六周就会有一个新的版本诞生，伴随着很多新的功能。比如 const generics（代码）：

复制代码

```
1 #[derive(Debug)]
```

```rust
struct Packet<const N: usize> {
    data: [u8; N],
}

fn main() {
    let ip = Packet { data: [0u8; 20] };
    let udp = Packet { data: [0u8; 8] };

    println!("ip: {:?}, udp: {:?}", ip, udp);
}
```

再比如最近刚发的 1.55 支持了 open range pattern（ 代码 ）：

复制代码

```rust
fn main() {
    println!("{}", match_range(10001));
}

fn match_range(v: usize) -> &'static str {
    match v {
        0..=99 => "good",
        100..=9999 => "unbelievable",
        10000.. => "beyond expectation",
        _ => unreachable!(),
    }
}
```

再过一个多月，Rust 就要发布 2021 edition 了。由于 Rust 良好的向后兼容能力，我建议保持使用最新的 Rust 版本。等 2021 edition 发布后，我会更新代码库到 2021 edition，文稿中的相应代码也会随之更新。

思考题

来一道简单的思考题，我们把之前学的内容融会贯通一下，代码展示了有问题的生命周期，你能找到原因么？（ 代码 ）

复制代码

```rust
use std::str::Chars;

// 错误，为什么？
fn lifetime1() -> &str {
    let name = "Tyr".to_string();
    &name[1..]
```

```rust
}

// 错误，为什么？
fn lifetime2(name: String) -> &str {
    &name[1..]
}

// 正确，为什么？
fn lifetime3(name: &str) -> Chars {
    name.chars()
}
```

欢迎在留言区抢答，也非常欢迎你分享这段时间的学习感受，一起交流进步。我们下节课回归正文讲 Rust 的类型系统，下节课见！

分享给需要的人，Ta订阅后你可得 20 元现金奖励

赞 7 提建议

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 11 | 内存管理：从创建到消亡，值都经历了什么？

下一篇 12 | 类型系统：Rust的类型系统有什么特点？

精选留言 (15)

写留言



lisiur

2021-09-17

第一个，没有标注生命周期，但即使标注也不对，因为返回值引用了本地已经 drop 的 String，会造成悬垂指针问题；

第二个，和第一个类似，因为参数是具有所有权的 String，该 String 会在函数执行完后被 drop，返回值不能引用该 String；...

展开

作者回复: 非常棒！

7



乌龙猹
2021-09-17

陈老师，啥时候再出一门 Elixir 编程的第一课啊

展开

作者回复: :) 做这一门课我就感觉已经竭尽全力了，做完估计要休半年才能缓过来

1 5



gnu
2021-09-17

lifetime1:
返回的引用是在 lifetime1 里被分配，lifetime1 结束后引用就被回收，所以错误。
改为转成 string 后返回。
```
fn lifetime1() -> String {...
```
展开

作者回复: 非常棒！修改得也很棒！

3



Arthur
2021-09-17

lifetime1:
name为函数内部的临时变量，类型是String，函数返回值为其引用，但引用的变量name生命周期在函数结束时，会被drop，因此此处引用失效，无值可借；

lifetime2:...

展开

作者回复: 非常好，目前最棒的答案！

1



罗杰
2021-09-17

比较简单的问题，第一个 name 在函数里面创建的 String，函数返回时就释放掉了，这是最直白的悬垂引用。第二个 name 是从调用者 move 过来的 String，进入该函数，所有权就归函数了，返回时 name 也将被释放。第三个 name 不用加生命周期标注可以正常工作，参数是引用，返回的数据与该参数的生命周期相同，没有问题，可以编译通过。

展开

作者回复: 非常棒！

1



记事本
2021-09-19

假如 泛型T约束需要实现AsRef trait, str 已经实现AsRef tarit,那么&str符合这个泛型参数吗?



Kerry
2021-09-18

例子一：

1. &str生命周期不明确
2. 返回了局部函数拥有所有权的引用，也是生命周期问题

...

展开

作者回复: 非常好。要注意的是例子 1 的改法并不是通用的，它只适合少数可以 &'static 的情况。

大哉乾元
2021-09-18

请教老师一个问题，关于文件操作的相对路径，如果是先编译再执行，rust会以可执行文件所在目录作为当前目录进行文件操作，如果直接cargo run的话则是以源文件目录作为相对目录执行，有办法统一起来么？

展开

作者回复: 不能。编译时和运行时是两个状态。cargo run 相当于在当前目录下运行编译出来的可执行文件，你也可以在任何目录下运行这个可执行文件。所以运行时不要假定程序运行的工作目

录。





记事本
2021-09-17

老师，关于智能指针一些问题：
数据放在堆上，返回指针给栈上的结构体
智能指针有个特点，*解耦到原型，&*就是获取数据的引用，单&栈上结构体的地址
*因为会解耦出原型，所以原数据是否实现copy trait，否则会move，智能指针就没有所有权了

展开

作者回复: 是的，所以 * 不能直接使用在诸如 String / Vec 这样数据结构的引用上。当你尝试这样做，编译器会报错：数据被 borrow，但又被 move，且没有实现 copy，所以错误。

```rust
fn main() {
    let s = "hello".to_string();
    let r1 = &s;
    let s1 = *r1;

    println!("{:?}", s1);
}
```

所以这里会编译出错，而不是移动所有权。你不能一边借用，一边移动所有权。






芥末小龙
2021-09-17

在用rust重新撸leetcode，真痛苦。

展开

作者回复: leetcode 跟纯算法有关，跟语法反倒关系不大吧。应该不需要和借用/生命周期打交道，也基本上不涉及太多所有权（除了 String 和 Vec<T>，基本上都是用各种数值类型吧）。


1




记事本
2021-09-17

老师，String，Vec算是智能指针吗？*String解除str，然后&*String就是&str了，Box::new()好像也可以这样用，Box::new(String::new)这样的使用，内存发生了什么变化啊

作者回复: String / Vec 都是智能指针。理论上任何实现了 Deref / DerefMut / Drop 的数据结构，都是智能指针。






Hurt
2021-09-17

打卡 真的是愚昧之巅了 需要重头再来了

展开

作者回复: 加油！






25ma
2021-09-17

1.返回是一个&str,但是name的生命周期在函数执行结束就已经drop掉了，所以会造成悬垂指针的问题
2.同样也是犯规，一个&str,不同的地方是当name这个不可变变量传递进函数lifttime2的时候已经将值move,然而这时再返回一个&str,同样也会造成悬垂指针的问题

展开

作者回复: 对！






Do
2021-09-17

太慢了

展开

编辑回复: 啥太慢了，更新速度嘛？现在专栏都是一周更新三篇，因为也要考虑到老师高质量输出的频率和大家学习的频率。如果觉得慢，说明你的的Rust进度非常不错，学有余力嘛，推荐配套

第一篇加餐里的资料来学习噢💪



hughieyu
2021-09-17

1. name drop了
2. name drop了
3. name的内存数据拷贝并转换到了一个新的拥有所有权的对象中

作者回复: 对！